PONTIFICIA UNIVERSIDAD CATÓLICA DEL PERÚ

FACULTAD DE CIENCIAS E INGENIERÍA

ALGORITMIA Y ESTRUCTURA DE DATOS

Primer Examen
(Primer Semestre 2026)

Duración: 2h 50 min.

- **No puede utilizar apuntes, solo hojas sueltas en blanco.**
- En cada función el alumno deberá incluir, a modo de comentario, la forma de solución que utiliza para resolver el problema. De no incluirse dicho comentario, el alumno perderá el derecho a reclamo en esa pregunta.
- No puede emplear plantillas o funciones no vistas en los cursos de programación de la especialidad.
- Los programas deben ser desarrollados en el lenguaje C++. Si la implementación es diferente a la estrategia indicada o no la incluye, la pregunta no será corregida.
- Un programa que no muestre resultados coherentes y/o útiles será corregido sobre el 50% del puntaje asignado a dicha pregunta.
- Debe utilizar comentarios para explicar la lógica seguida en el programa elaborado. El orden será parte de la evaluación.
- **Solo está permitido acceder a la plataforma de PAIDEIA, cualquier tipo de navegación, búsqueda o uso de herramientas de comunicación se considera plagio por tal motivo se anulará la evaluación y se procederá con las medidas disciplinarias dispuestas por la FCI.**
- Para esta evaluación solo se permite el uso de las librerías **iostream, iomanip, climits cstring, cmath o fstream**
- Su trabajo deberá ser subido a PAIDEIA.
- **Es obligatorio usar como IDE CLion.**
- Los archivos deben llevar como nombre su código de la siguiente forma codigo_EX1_P# (donde # representa el número de la pregunta a resolver)

Elija las preguntas para obtener un puntaje de 20, si escoge más de esa nota se considera la respuesta con más bajo puntaje:

Pregunta 1 (5 puntos)

El área de seguridad de la información de una empresa registra el tráfico saliente de su red con la finalidad de identificar posibles patrones de exfiltración de datos[1]. Para ello, almacena la información en una matriz tráfico **FxC**, donde cada fila representa un minuto de monitoreo y cada columna representa un destino externo. Cada celda contiene la cantidad de MB enviados en ese minuto hacia ese destino.

Se requiere implementar un programa en C++ que, usando **fuerza bruta**, analice todos los bloques rectangulares posibles dentro de la matriz y reporte aquellos cuya suma total de tráfico sea mayor que un umbral sospechoso **U**.

Cada bloque rectangular debe cumplir lo siguiente:

- ✓ Su altura debe ser como máximo **H**.
- ✓ Su ancho debe ser como máximo **W**.
- ✓ La suma de sus valores debe ser mayor que **U**.

---

[1] **Exfiltración de datos**: Es la extracción y transferencia no autorizada de información sensible desde un sistema hacia un destino externo.

El programa debe mostrar todos los bloques sospechosos encontrados, indicando:

i. Fila inicial
ii. Columna inicial
iii. Fila final
iv. Columna final
v. Suma total del bloque

Si no se encuentra ningún bloque sospechoso, el programa debe mostrar: "No se encontraron patrones sospechosos". El programa debe trabajar únicamente con una matriz de enteros. A continuación, se presentan 3 ejemplos de ejecución.

Ejemplo de ejecución 1:

Datos de entrada:
F = 4; C = 4; H = 2; W = 2; U = 50;
trafico[][] = {{1, 2, 3, 4}, {5, 20, 1, 2}, {3, 25, 4, 1}, {1, 2, 30, 3}};

Salida esperada:
Matriz evaluada:

1	2	3	4
5	20	1	2
3	25	4	1
1	2	30	3

Altura máxima del bloque: 2
Ancho máximo del bloque: 2
Umbral sospechoso: 50

Bloque sospechoso 1:
Fila inicial: 1
Columna inicial: 0
Fila final: 2
Columna final: 1
Suma: 53

Bloque sospechoso 2:
Fila inicial: 2
Columna inicial: 1
Fila final: 3
Columna final: 2
Suma: 61

Ejemplo de ejecución 2:

Datos de entrada:
F = 5; C = 6; H = 3; W = 2; U = 80;
trafico[][] = {{4, 1, 3, 2, 5, 1}, {10, 35, 2, 1, 4, 3}, {8, 30, 5, 2, 6, 4}, {1, 25, 20, 3, 2, 1}, {2, 1, 15, 40, 3, 2}};

Salida esperada:
Matriz evaluada:

4	1	3	2	5	1
10	35	2	1	4	3
8	30	5	2	6	4
1	25	20	3	2	1
2	1	15	40	3	2

Altura máxima del bloque: 3
Ancho máximo del bloque: 2
Umbral sospechoso: 80

Bloque sospechoso 1:
Fila inicial: 0
Columna inicial: 0
Fila final: 2
Columna final: 1

Suma: 88

Bloque sospechoso 2:
Fila inicial: 1
Columna inicial: 0
Fila final: 2
Columna final: 1
Suma: 83

Bloque sospechoso 3:
Fila inicial: 1
Columna inicial: 0
Fila final: 3
Columna final: 1
Suma: 109

Bloque sospechoso 4:
Fila inicial: 1
Columna inicial: 1
Fila final: 3
Columna final: 1
Suma: 90

Bloque sospechoso 5:
Fila inicial: 1
Columna inicial: 1
Fila final: 3
Columna final: 2
Suma: 117

Bloque sospechoso 6:
Fila inicial: 2
Columna inicial: 1
Fila final: 4
Columna final: 2
Suma: 96

Bloque sospechoso 7:
Fila inicial: 2
Columna inicial: 2
Fila final: 4
Columna final: 3
Suma: 85

Importante*: En este ejemplo de ejecución aparece también un bloque de una sola columna, por ejemplo, el* ***bloque 4****, porque el código prueba anchos desde 1 hasta W. Es decir, con W = 2, evalúa bloques de ancho 1 y de ancho 2.*

Ejemplo de ejecución 3:

Datos de entrada:
F = 3; C = 3; H = 2; W = 2; U = 50;
trafico[MAX][MAX] = {{1, 2, 3}, {4, 5, 6}, {7, 8, 9}};

Salida esperada:
Matriz evaluada:
1 2 3
4 5 6
7 8 9

Altura máxima del bloque: 2
Ancho máximo del bloque: 2
Umbral sospechoso: 50

No se encontraron patrones sospechosos

Pregunta 2 (5 puntos)

En procesamiento de lenguaje natural (NLP), una de las tareas más básicas consiste en analizar la frecuencia de aparición de palabras dentro de un texto. Este tipo de procesamiento es utilizado en motores de búsqueda, chatbots, etc. Para este ejercicio, implemente una solución **completamente recursiva** que permita contar cuántas veces aparece una palabra dentro de un texto.

Problema

Implemente una función recursiva que reciba: un arreglo de caracteres que representa un **texto** y un arreglo de caracteres que representa una **palabra** y retorne la cantidad de veces que dicha palabra aparece dentro del texto.

Consideraciones importantes

- Las palabras del texto están separadas únicamente por espacios.
- Debe contarse únicamente coincidencias completas de palabra.
- No deben contarse coincidencias parciales.
- Puede agregar los parámetros que vea necesarios en las funciones recursivas.

Ejemplo

TEXTO	PALABRA	SALIDA
hola mundo hola amigos hola	hola	3
hola mundo hola holanda	hola	2

Restricciones

La solución debe cumplir obligatoriamente con lo siguiente:

- No usar estructuras iterativas: for, while, do while.
- No usar: STL, string, arreglos auxiliares, funciones de librería para búsqueda de texto.
- Toda la solución debe implementarse mediante recursividad pura.

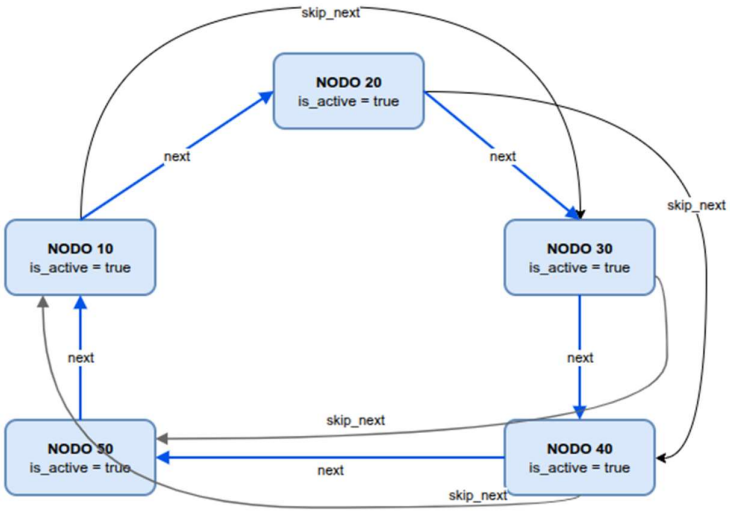
Pregunta 3 (10 puntos)

En un sistema de almacenamiento distribuido, los nodos se organizan en una estructura de **lista circular** o anillo. Para garantizar que los mensajes lleguen a su destino incluso si un nodo colapsa en medio de la transmisión, cada nodo del clúster mantiene dos canales de comunicación salientes:

- Un puntero al nodo siguiente inmediato (next).
- Un puntero de respaldo que salta al subsiguiente (skip_next), es decir, al vecino de su vecino.

Implemente en C++ el TAD bajo las siguientes restricciones estrictas de diseño:

- Cada nodo representa un proceso y debe contener:
    - int node_id
    - bool is_active
- Punteros next y skip_next.



Prohibido usar la STL o estructuras auxiliares (como arreglos temporales). Se permite el uso de la BibliotecaLista.

a. (0.5) Defina la estructura del TAD en base a la descripción proporcionada.
b. (1.5) Implemente la función **agregar_nodo**. Esta función agrega un nodo al anillo y siempre lo mantiene cerrado **(actualiza solo el atributo next)**.
c. (1.5) Implemente la función **calcular_skip_next**. Esta función recorre el anillo ya formado y calcula/actualiza todos los punteros skip_next.
d. (4.0) Implemente una función void **auto_healing**. Cuando un nodo se cae, el sistema sigue funcionando gracias a skip_next, pero el anillo queda debilitado. Este método debe recorrer todo el anillo y **reconfigurar permanentemente los punteros de los nodos activos para aislar por completo a los nodos muertos (Ver imagen debajo)**.
   i. Si el next de un nodo está muerto, su nuevo next debe ser el siguiente nodo activo.
   ii. El puntero skip_next también debe actualizarse para referenciar al siguiente nodo activo que le sigue al nuevo nodo next calculado en el punto (i).
   iii. Los nodos caídos deberán ser eliminados. Si el nodo inicial del anillo esta caído, este deberá ser reemplazado por el próximo activo en caso existiese.
   El algoritmo debe ser capaz de lidiar con dos o más nodos caídos de forma consecutiva.
e. (0.5) Implemente la función **desactivar_nodo**. Esta función, dado el id de un nodo, cambia el valor de is_active a false.
f. (2.0) Implemente la función **main** la cual recibe como dato de ingreso un N = 5 y luego:
   a. Crea 5 nodos. Cada nodo tendrá un id múltiplo de 10**.**
   b. Imprime el estado inicial del anillo
   c. Desactiva los nodos 20 y 30
   d. Imprime el estado del anillo antes del auto_healing.
   e. Imprime el estado del anillo después del auto_healing.

Datos de entrada: N = 5
Salida esperada: === Estado inicial === Anillo (5 nodos): Nodo 10 [ACTIVO] siguiente=20 skip_next=30 Nodo 20 [ACTIVO] siguiente=30 skip_next=40

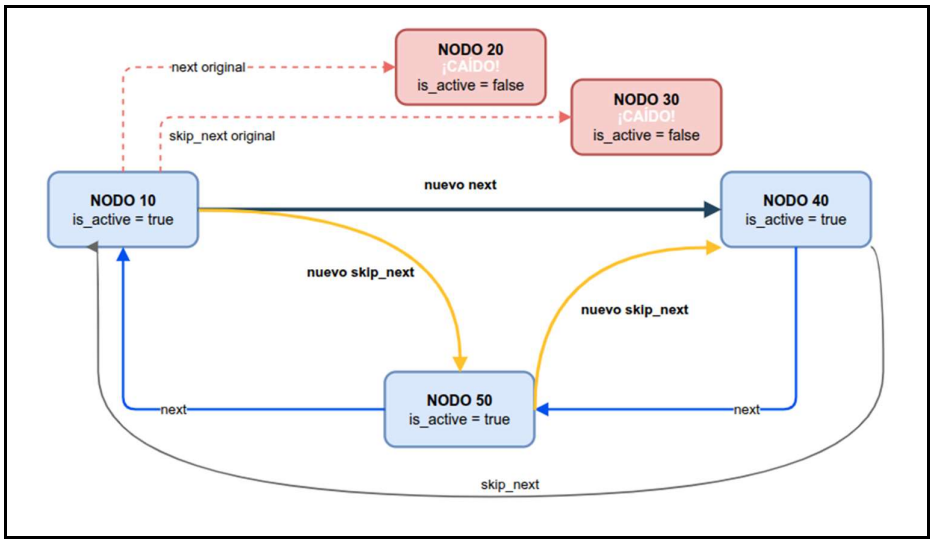
```
  Nodo 30 [ACTIVO]  siguiente=40  skip_next=50
  Nodo 40 [ACTIVO]  siguiente=50  skip_next=10
  Nodo 50 [ACTIVO]  siguiente=10  skip_next=20

>>> Nodo 20 cae <<<

>>> Nodo 30 cae <<<

=== Estado antes de auto_healing ===
Anillo (5 nodos):
  Nodo 10 [ACTIVO]  siguiente=20  skip_next=30
  Nodo 20 [CAIDO ]  siguiente=30  skip_next=40
  Nodo 30 [CAIDO ]  siguiente=40  skip_next=50
  Nodo 40 [ACTIVO]  siguiente=50  skip_next=10
  Nodo 50 [ACTIVO]  siguiente=10  skip_next=20

=== Estado después de auto_healing ===
Anillo (3 nodos):
  Nodo 10 [ACTIVO]  siguiente=40  skip_next=50
  Nodo 40 [ACTIVO]  siguiente=50  skip_next=10
  Nodo 50 [ACTIVO]  siguiente=10  skip_next=40
```



Pregunta 4 (5 puntos)

Una empresa registra los accesos de sus empleados a una zona restringida mediante una secuencia de caracteres.

Cada vez que un empleado **ingresa**, se registra una letra mayúscula que representa su código. Cada vez que un empleado **sale**, se registra la misma letra, pero en minúscula.

Por ejemplo:

Abba

Esta secuencia representa:

A ingresa

B ingresa

b sale

a sale

La secuencia es válida porque el último empleado que ingresó fue el primero en salir. Implemente un programa en C++ que, utilizando **solo una pila**, determine si una secuencia de accesos es válida. Una secuencia será válida si:

- Cada salida corresponde al último ingreso pendiente.
- No puede salir un empleado que no haya ingresado.
- Al finalizar, no deben quedar empleados dentro de la zona restringida.

Por ejemplo:
Secuencia: ABba
Salida: Secuencia válida

Secuencia: ABabCcAa
Salida: Secuencia inválida

Secuencia: ABbaCcAa
Salida: Secuencia válida

Nota. Debe realizar una solución general y no solo para los casos colocados como ejemplo.

Pregunta 5 (5 puntos)

Generador de números binarios: Usando solo **una cola**, genere los primeros N números binarios en orden ascendente 1, 10, 11, 100, 101, 110, 111, 1000, 1001, 1010, …
El único dato de entrada es N. Debe encontrar la relación que existe entre un número y los que siguen. Es importante que tome en cuenta las siguientes restricciones para la calificación:

- El único tipo de dato permitido es **int** (o **long long** si N es grande)
- No se permite convertir un número decimal a binario, ni directamente ni con funciones de biblioteca.
- La cola debe ser la única estructura auxiliar utilizada.
- Use el TAD de cola brindada.
- La función **main** debe tener únicamente la lectura de N y la llamada a la función que genere los números.
- Cree una carpeta llamada **Bibliotecas** dónde guarde los archivos cpp y h
- La función que genera los números debe encontrarse dentro de la carpeta Bibliotecas, use el nombre que crea conveniente para dicha función.

Ejemplos:
Para N=3, => 1, 10, 11
Para N=6, => 1, 10, 11, 100, 101, 110

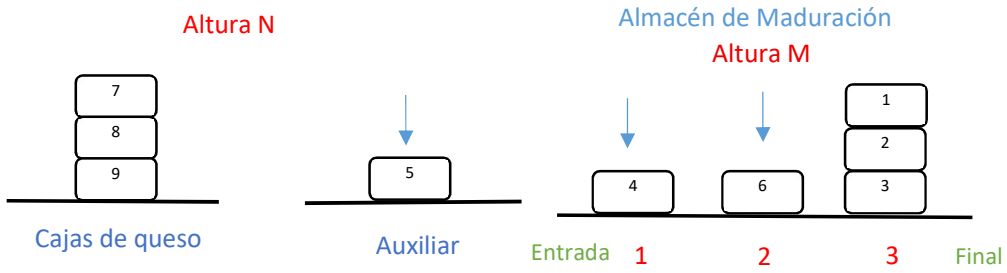
Criterios de calificación	**Puntos**
Encuentra la relación que existe entre un número y los que siguen	2 puntos
Usa la cola correctamente: encola, desencola	2 puntos
Cumple las restricciones de implementación y del enunciado	1 punto

Pregunta 6 (10 puntos)

La fábrica de quesos "Vaquita Feliz" necesita trasladar las cajas de quesos recién producidos hacia los almacenes de maduración. Para automatizar esta tarea, la empresa adquiere un robot modelo H-Milk, cuya programación presenta restricciones particulares: su memoria está limitada al uso de pilas y variables enteras. Las cajas de quesos se encuentran inicialmente organizadas en una pila de altura N, ubicada fuera del almacén. Cada caja posee un peso distinto y, para evitar daños en los productos, las cajas más pesadas deben permanecer siempre debajo de las más ligeras.

El almacén de maduración dispone internamente de tres espacios de apilamiento, cada uno con capacidad máxima de M cajas, además de un espacio auxiliar ubicado en la entrada del almacén. Se sabe que N > M. El proceso de traslado debe respetar en todo momento las siguientes restricciones e indicaciones:

a. Solo puede moverse una caja a la vez.
b. Nunca puede colocarse una caja más pesada sobre una más ligera.
c. Debe llenarse primero la pila más interna del almacén de maduración para esa tarea deben emplearse de forma obligatoria la pila auxiliar y los dos espacios de apilamiento que tiene el almacén de maduración, a parte de la pila origen y destino. Este proceso se debe realizar en una sola función recursiva.
d. Luego debe llenarse la segunda pila más interna del almacén, para esa tarea debe emplearse de forma obligatoria la pila auxiliar y el espacio de apilamiento libre que tiene el almacén, a parte de la pila origen y destino. Este proceso se debe realizar en una sola función recursiva diferente a la anterior.
e. Finalmente, las cajas restantes deben moverse de al espacio de apilamiento libre empleando la pila auxiliar, a parte de la pila origen y destino. Este proceso se debe realizar en una sola función recursiva diferente a las dos anteriores.



A continuación, algunos ejemplos:

Ingreso: N=9 y M=3 **Resultado:** Pila 3:[1, 2, 3] Pila 2:[4, 5, 6] Pila 1:[7, 8, 9]	**Ingreso:** N=5 y M=3 **Resultado:** Pila 3:[1, 2, 3] Pila 2:[4, 5] Pila 1: Pila vacía	**Ingreso:** N=10 y M=4 **Resultado:** Pila 3:[1, 2, 3, 4] Pila 2:[5, 6, 7, 8] Pila 1:[9, 10]

Se necesita que se desarrolle un algoritmo en C++ que permita distribuir correctamente todas las cajas dentro del almacén, respetando las restricciones indicadas. **No puede emplear ninguna estructura (arreglos, listas, colas, pilas) adicional a las indicadas en el problema. En caso no se respete lo indicado la respuesta no tendrá puntaje. No se aceptan soluciones parciales debe mostrar el resultado completo.**

Criterios de calificación	Puntos
Realiza los movimientos con la función de la parte e.	0.5 puntos
Realiza los movimientos con la función de la parte d.	1.5 puntos
Realiza los movimientos con la función de la parte c.	8.0 puntos

Al finalizar el examen, comprima la carpeta de su proyecto empleando el programa Zip que viene por defecto en el Windows, **no se aceptarán los trabajos compactados con otros programas como RAR, WinRAR, 7zip o similares**. Luego súbalo a la tarea programa en Paideia para este examen.

Profesores del curso:

Ana Roncal
Fernando Huamán
David Allasi
Erasmo Gómez
José Luis Corcuera
Rony Cueva

San Miguel, 23 de mayo del 2026